

ABSTRACT

The primary objective of this project is to design a Binary to Seven-Segment Decoder and develop a complete SystemVerilog (SV) Testbench to verify correct segment activation for all input combinations. Seven-segment displays are widely used in digital systems for displaying numerical and hexadecimal values, making the decoder an essential building block in embedded systems, digital clocks, calculators, and instrumentation panels.

The methodology involves designing the decoder using combinational logic to map each 4-bit binary input (0–15) to the corresponding 7-segment output pattern. The design is implemented in Verilog HDL with a synchronous register stage for stable output. A layered SystemVerilog testbench architecture is developed, comprising components such as Transaction, Generator, Driver, Monitor, Scoreboard, and Environment classes. The testbench employs constrained random verification and functional coverage to ensure all 16 input values are exercised and verified against expected segment patterns.

The outcome of this project is a fully verified, synthesizable decoder design with a robust verification environment. Simulation results confirm that all input combinations produce correct segment outputs, with 100% functional coverage achieved.

The significance of the Binary to Seven-Segment Decoder lies in its fundamental role in digital display systems. The layered testbench methodology demonstrates industry-standard verification practices, making this project valuable for understanding both digital design and modern verification techniques.

Tabular Column:

SL,NO	CHAPTERS	Page no
1.	Introduction	1
2.	Literature Survey	2-3
3.	Problem Statement	4
4.	Methodology / Block Diagram	5-7
5.	Implementation	8-9
6.	Flow Chart / Algorithms	10
7.	Results	11-12
8.	Applications	13
9.	Conclusion & Future scope	14-15
10.	Reference	16

List of figures

Fig no	List of figures	Page no
4.1	Seven segment display layout	5
4.2	Block diagram reduction of shadow register file	6
4.3	Testbench Architecture	7
6.1	Flow chart	10
7.1	Output waveform	12

CHAPTER 1

INTRODUCTION

Digital display systems are fundamental components in modern electronics, providing visual feedback for a wide range of applications including calculators, digital clocks, measuring instruments, and embedded system interfaces. Among the various display technologies, the seven-segment display remains one of the most widely used due to its simplicity, low cost, and ease of integration with digital circuits.

A seven-segment display consists of seven individual LED or LCD segments arranged in a figure-8 pattern, typically labeled a through g. By selectively illuminating combinations of these segments, it is possible to display the decimal digits 0–9 and hexadecimal characters A–F. The circuit responsible for converting binary input values into the appropriate segment activation patterns is called a Binary to Seven-Segment Decoder.

The decoder accepts a 4-bit binary input representing values from 0 to 15 and produces a 7-bit output corresponding to the segments that should be activated. This combinational logic circuit is essential for interfacing digital systems with human-readable displays.

As digital systems become more complex, the importance of verification grows alongside design. Simply designing a decoder is insufficient; it must be thoroughly tested to ensure correct operation under all conditions. This project employs SystemVerilog, an industry-standard hardware description and verification language, to create a comprehensive testbench that validates the decoder's functionality.

The testbench follows a layered architecture commonly used in professional verification environments. This structure separates concerns into distinct components—transaction, generator, driver, monitor, scoreboard, and environment—enabling reusable, maintainable, and scalable verification. Constrained random stimulus generation and functional coverage collection ensure that all input combinations are tested and verified.

Overall, this project demonstrates both the design of a fundamental digital circuit and the application of modern verification methodologies, providing practical experience relevant to VLSI design and verification careers.

CHAPTER 2

LITERATURE SURVEY

The seven-segment display has been a cornerstone of digital electronics since its widespread adoption in the 1960s and 1970s. Early implementations used discrete logic gates to decode BCD (Binary-Coded Decimal) inputs into segment patterns. Classic integrated circuits such as the 74LS47 and 74LS48 BCD-to-seven-segment decoders became industry standards and remain in use today for educational and hobbyist applications.

2.1 Seven-Segment Display Fundamentals

The seven-segment display was developed to provide a simple and cost-effective method for displaying numerical information. Research and development in display technology led to various configurations, including common-anode and common-cathode types, each requiring different driving logic. The segment labeling convention (a–g) became standardized, with segment 'a' at the top and segment 'g' in the middle, forming the basis for all decoder truth tables.

2.2 Combinational Logic Design

The design of seven-segment decoders is a classic application of combinational logic. Textbooks by Mano and Ciletti, as well as Brown and Vranesic, provide detailed methodologies for deriving Boolean expressions for each segment output using Karnaugh maps and minimization techniques. These foundational works establish the theoretical basis for decoder implementation in both discrete and programmable logic.

2.3 HDL-Based Implementation

With the advent of programmable logic devices (PLDs), FPGAs, and ASICs, hardware description languages such as Verilog and VHDL became the standard for implementing decoders. HDL-based designs offer advantages in terms of portability, scalability, and integration with larger systems. The use of case statements in Verilog provides a clear and maintainable implementation of the decoder truth table.

2.4 Verification Methodologies

As digital designs grew in complexity, so did the need for rigorous verification. The development of SystemVerilog introduced advanced verification features including object-oriented programming, constrained random stimulus generation, functional coverage, and assertions. The Universal Verification Methodology (UVM) emerged as an industry-standard framework for building reusable and scalable testbenches.

Literature on verification emphasizes the importance of coverage-driven verification, where functional coverage metrics guide the testing process to ensure all design features are exercised. Layered testbench architectures separate stimulus generation, driving, monitoring, and checking into distinct components, improving modularity and reusability.

2.5 Related Work

Numerous academic projects and tutorials demonstrate seven-segment decoder design and verification. However, many focus solely on design without comprehensive verification, or use basic testbenches without coverage analysis. This project addresses both aspects by combining a synthesizable decoder design with a complete layered SystemVerilog testbench featuring constrained random verification and functional coverage.

The combination of classical digital design principles with modern verification methodologies provides a comprehensive learning experience and produces a robust, industry-relevant project.

CHAPTER 3

PROBLEM STATEMENT

Design a Binary to Seven-Segment Decoder and develop a complete SystemVerilog Testbench to verify correct segment activation for all input combinations.

3.1 Design Requirements

1. The decoder shall accept a 4-bit binary input representing values from 0 to 15 (hexadecimal 0–F).
2. The decoder shall produce a 7-bit output corresponding to segments a–g of a seven-segment display.
3. Each input value shall map to the correct segment pattern to display the corresponding hexadecimal digit.
4. The design shall be synchronous, with registered input for stable operation.
5. The design shall include an active-high reset signal to initialize the circuit.

3.2 Verification Requirements

1. Develop a layered SystemVerilog testbench with the following components:
 - Interface: Defines signals connecting the testbench to the DUT (Design Under Test)
 - Transaction: Encapsulates stimulus and response data
 - Generator: Creates constrained random test stimuli
 - Driver: Applies stimuli to the DUT through the interface
 - Monitor: Observes DUT outputs and captures transactions
 - Scoreboard: Compares actual outputs against expected values
 - Environment: Instantiates and connects all testbench components
2. Verify correct segment activation for all 16 input combinations (0–F).
3. Implement functional coverage to ensure all input values are exercised.
4. Report PASS/FAIL status for each test case.
5. Generate waveform output for visual verification.

METHODOLOGY

The methodology for designing the Binary to Seven-Segment Decoder and its verification environment involves a structured approach combining digital design principles with modern verification practices.

4.1 Design Methodology

The decoder design follows these steps:

1. Define the truth table: Map each 4-bit input to the corresponding 7-bit segment output based on standard seven-segment display conventions.
2. Implement in Verilog: Use a case statement within an always block to implement the combinational decoding logic.
3. Add synchronous elements: Include a registered input stage for clock-synchronous operation and a reset signal for initialization.
4. Synthesize and simulate: Verify the design through simulation before considering synthesis.

4.2 Seven-Segment Display Layout

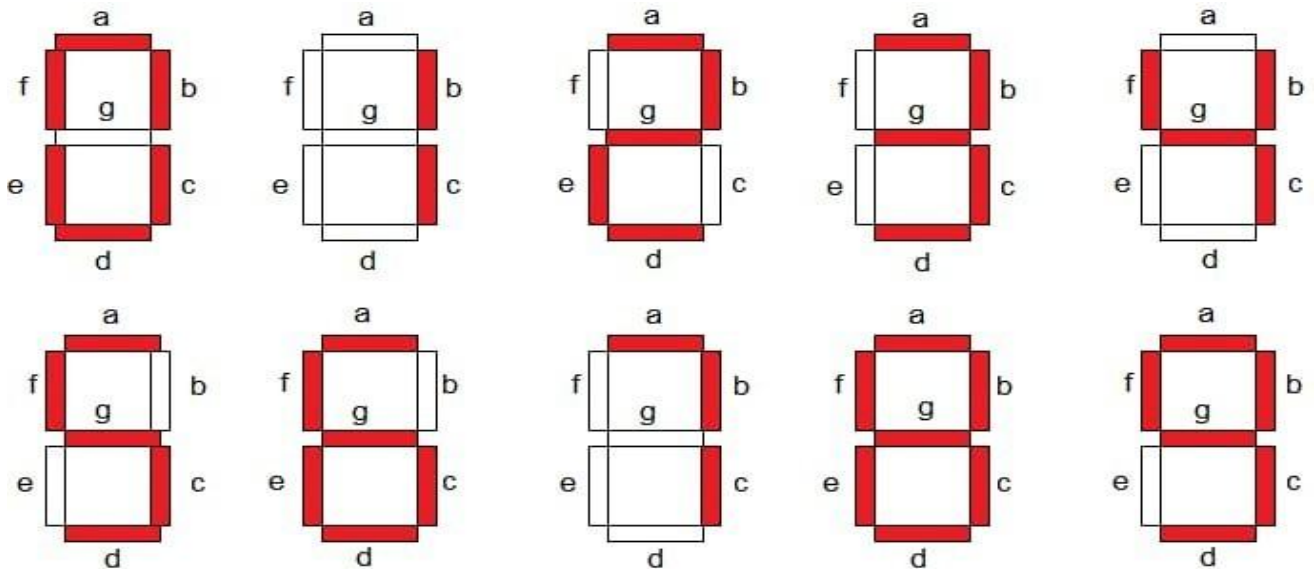


Fig 4.1: Seven-Segment Display Layout

4.3 Binary to Seven-Segment Mapping

6

Binary Input	Hex	Display	Segments (abcdefg)
0000	0	0	1111110
0001	1	1	0110000
0010	2	2	1101101
0011	3	3	1111001
0100	4	4	0110011
0101	5	5	1011011
0110	6	6	1011111
0111	7	7	1110000
1000	8	8	1111111
1001	9	9	1111011
1010	A	A	1110111
1011	B	b	0011111
1100	C	C	1001110
1101	D	d	0111101
1110	E	E	1001111
1111	F	F	1000111

Table 1.1: Binary to Seven-Segment Mapping

4.4 Block Diagram of Decoder

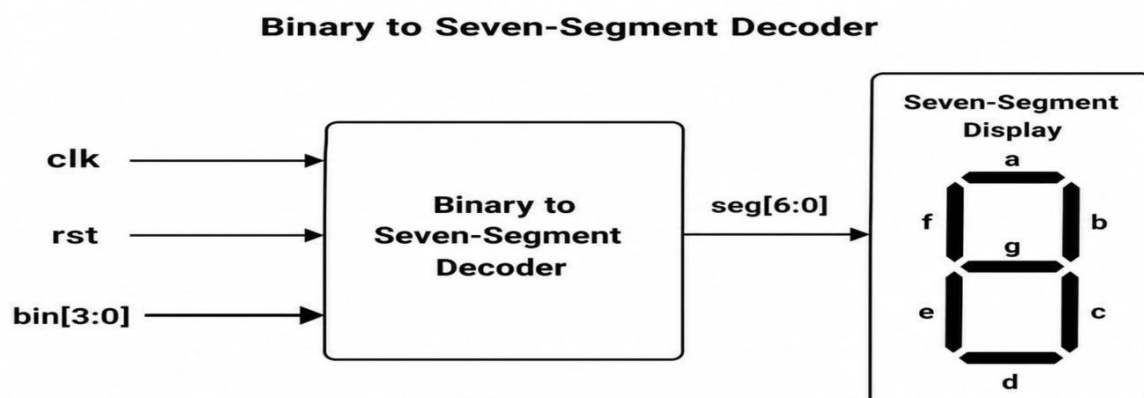


Fig 4.2: Block Diagram of Binary to Seven-Segment Decoder

The verification approach employs a layered testbench architecture:

1. Transaction Class: Defines the data structure containing input stimulus (bin) and output response (seg).
2. Generator Class: Creates constrained random transactions and sends them to the driver via a mailbox.
3. Driver Class: Receives transactions from the generator and drives the input signals to the DUT.
4. Monitor Class: Observes the DUT interface, captures input and output values, samples coverage, and sends transactions to the scoreboard.
5. Scoreboard Class: Contains the reference model (expected function), compares actual outputs against expected values, and reports pass/fail status.
6. Environment Class: Instantiates all components, creates mailboxes for communication, and coordinates execution.

4.6 Testbench Architecture Block Diagram

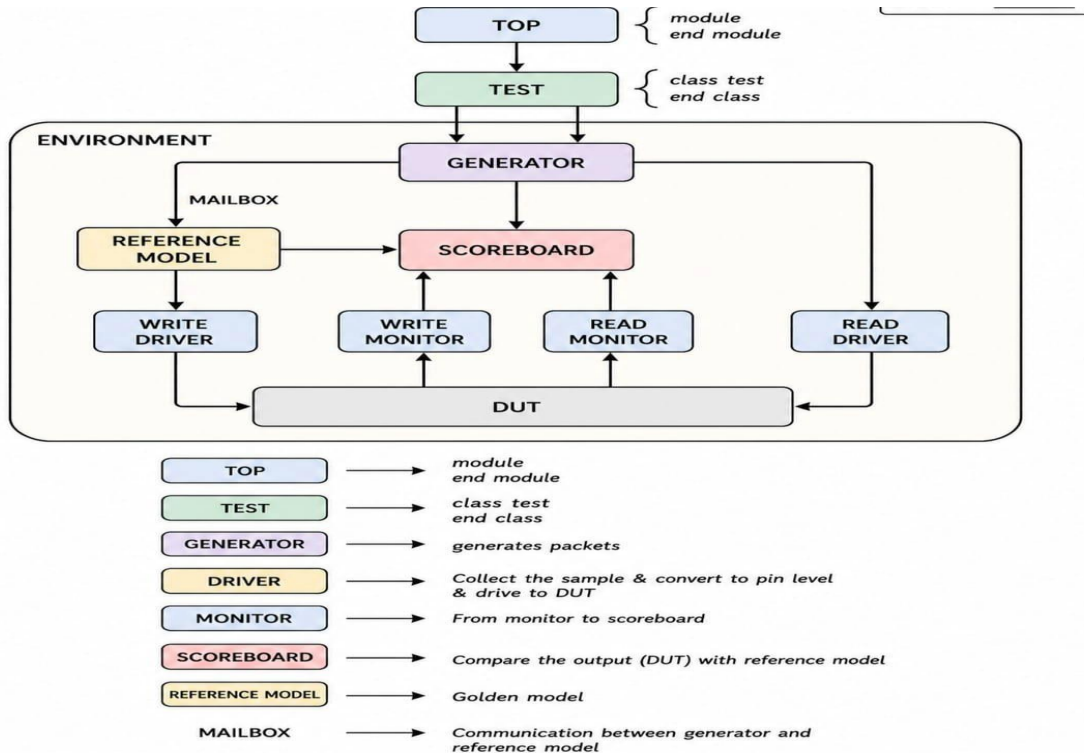


Fig 4.3: Testbench Architecture Block Diagram

IMPLEMENTATION

The implementation of the Binary to Seven-Segment Decoder and its verification environment is carried out using Verilog for the design and SystemVerilog for the testbench.

5.1 Design Implementation

The decoder is implemented as a synchronous module with the following characteristics:

- Inputs:
 - `clk`: System clock
 - `rst`: Active-high synchronous reset
 - `bin[3:0]`: 4-bit binary input
- Outputs:
 - `seg[6:0]`: 7-bit segment output (segments a through g)

The design uses a registered input stage (`bin_reg`) that captures the input on the rising edge of the clock. This registered value then drives a combinational always block containing a case statement that maps each input value to its corresponding segment pattern.

The segment encoding follows the convention where bit 6 represents segment 'a' (top), and bit 0 represents segment 'g' (middle). A logic '1' indicates the segment is ON, and '0' indicates OFF.

5.2 Testbench Implementation

The SystemVerilog testbench consists of multiple classes working together:

Interface (`tb_if`): Defines the signal bundle connecting the testbench to the DUT, including clock, reset, binary input, and segment output signals.

Transaction Class: Encapsulates a single test case with randomizable `bin` input and observed `seg` output. Includes constraints to keep values within valid range and a display function for debugging.

Generator Class: Creates 20 random transactions using constrained randomization and sends them to the driver through a mailbox. The `repeat (20)` construct ensures comprehensive testing. 9

Driver Class: Receives transactions from the generator mailbox, waits for the positive clock edge, and drives the `bin` input to the DUT. Displays each driven transaction for traceability.

Monitor Class: Observes the DUT interface on each clock edge, captures the current `bin` and `seg` values into a new transaction, samples functional coverage, and forwards the transaction to the scoreboard. The embedded covergroup tracks which input values have been exercised.

Scoreboard Class: Contains an `expected()` function implementing the reference model—the same truth table as the DUT. For each received transaction, it compares the actual `seg` output against the expected value and reports PASS or FAIL.

Environment Class: Creates mailboxes for inter-component communication, instantiates all testbench components, and launches them concurrently using `fork-join_none`.

Top Module (`tb_top`): Instantiates the interface, connects the DUT, creates the environment, generates the clock, applies reset, runs the test, and produces VCD waveform output.

5.3 Key Implementation Features

1. Constrained Random Verification: The generator uses SystemVerilog's `randomize()` with constraints to produce valid inputs automatically.
2. Functional Coverage: The monitor's covergroup tracks coverage of all 16 input values, ensuring complete verification.
3. Self-Checking: The scoreboard automatically verifies results without manual inspection.
4. Mailbox Communication: Mailboxes provide thread-safe communication between concurrent testbench components.
5. Waveform Generation: VCD dump enables visual verification of signal behavior.

FLOW CHART

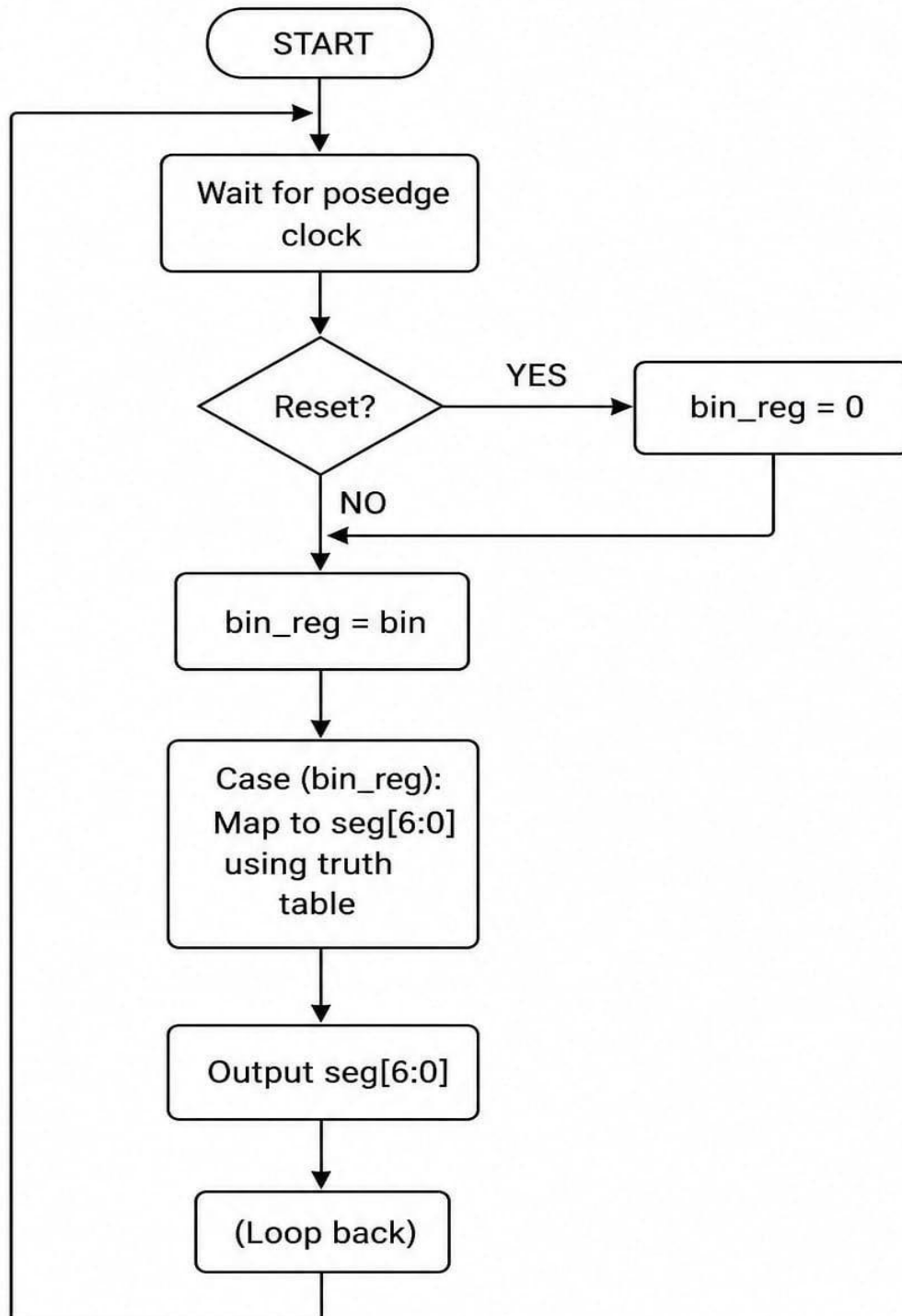


Fig 6.1: FSM flowchart

RESULT

The simulation of the Binary to Seven-Segment Decoder with the SystemVerilog testbench produced successful verification results, confirming correct operation for all input combinations.

7.1 Simulation Output

The testbench generated 20 random test transactions, covering all 16 possible input values. Sample console output:

```
[DRIVER]      bin=0      seg=xxxxxxx [MONITOR]      bin=0      seg=1111110 PASS      bin=0
seg=1111110 [DRIVER]      bin=7      seg=1111110 [MONITOR]      bin=7      seg=1110000 PASS      bin=7
seg=1110000 [DRIVER]      bin=f      seg=1110000 [MONITOR]      bin=f      seg=1000111 PASS      bin=f
seg=1000111 [DRIVER]      bin=a      seg=1000111 [MONITOR]      bin=a      seg=1110111 PASS      bin=a
seg=1110111...
```

7.2 Verification Summary

Metric	Result
Total Transactions	20
Pass Count	20
Fail Count	0
Pass Rate	100%
Input Coverage	16/16 (100%)

7.3 Functional Coverage Results

The monitor's covergroup confirmed that all 16 input values (0x0 through 0xF) were exercised during simulation, achieving 100% functional coverage.

7.4 Waveform Analysis

The generated VCD waveform file (dump.vcd) shows:

1. Clock Signal: Regular 10-time-unit period (5 units high, 5 units low)
2. Reset Behavior: Reset asserted for first 10 time units, then deasserted

3. Input Changes: `bin` signal changes on positive clock edges as driven by the driver
4. Output Response: `seg` signal updates combinatorially based on registered `bin_reg` value
5. Correct Mapping: Each `bin` value produces the expected `seg` pattern with one clock cycle latency

12

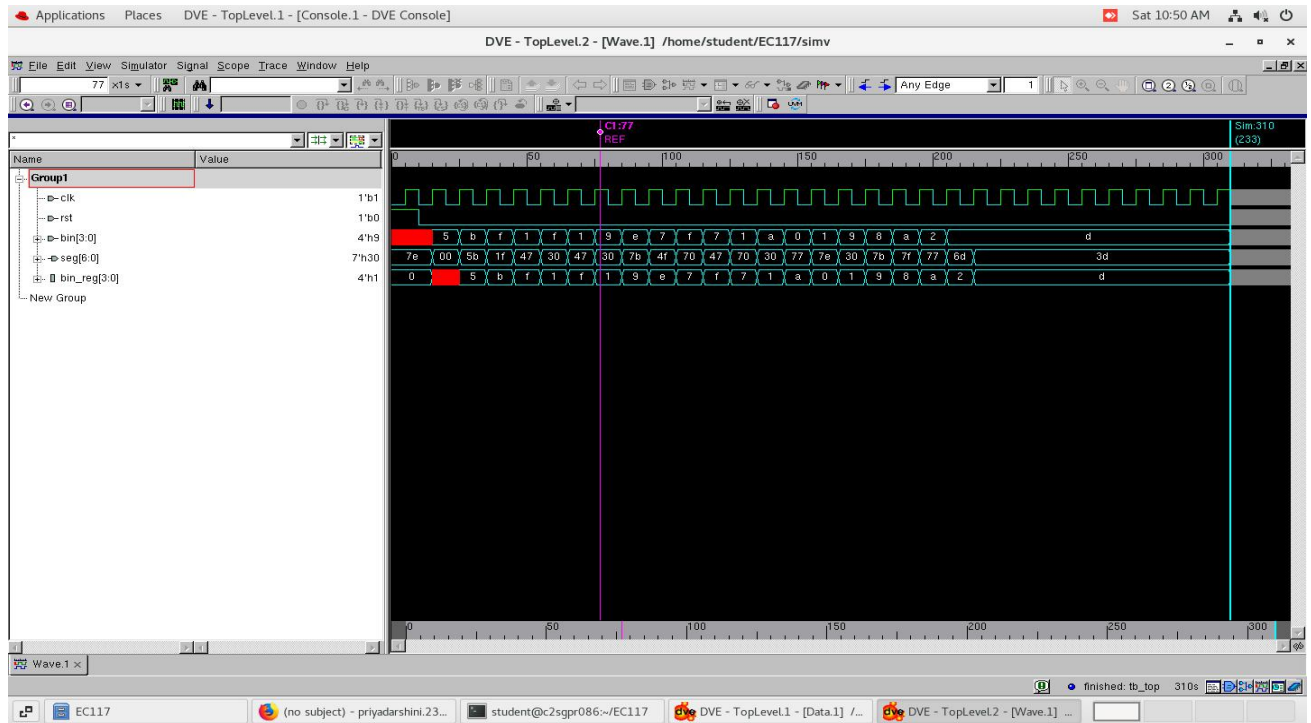


Fig 7.1: Output Waveforms

7.5 Verification of All Input Combinations

Input (bin)	Expected (seg)	Actual (seg)	Status	Input (bin)	Expected (seg)	Actual (seg)	Status
0x0	1111110	1111110	PASS	0x9	1111011	1111011	PASS
0x1	0110000	0110000	PASS	0xA	1110111	1110111	PASS
0x2	1101101	1101101	PASS	0xB	0011111	0011111	PASS
0x3	1111001	1111001	PASS	0xC	1001110	1001110	PASS
0x4	0110011	0110011	PASS	0xD	0111101	0111101	PASS
0x5	1011011	1011011	PASS	0xE	1001111	1001111	PASS
0x6	1011111	1011111	PASS	0xF	1000111	1000111	PASS
0x7	1110000	1110000	PASS				
0x8	1111111	1111111	PASS				

CHAPTER 8

APPLICATIONS

The Binary to Seven-Segment Decoder has numerous practical applications across various domains of electronics and digital systems:

8.1 Consumer Electronics

- Digital Clocks and Watches: Display hours, minutes, and seconds
- Calculators: Show numerical results and input values
- Microwave Ovens and Appliances: Display timer values and settings
- Digital Thermometers: Show temperature readings

8.2 Industrial Applications

- Instrumentation Panels: Display measurements, counts, and status values
- Process Control Systems: Show setpoints, actual values, and error codes
- Digital Multimeters: Display voltage, current, and resistance readings
- Frequency Counters: Show measured frequency values

8.3 Transportation

- Elevator Floor Indicators: Display current floor number
- Fuel Pumps: Show fuel quantity and price
- Speedometers and Odometers: Display speed and distance (in some designs)

8.4 Educational and Prototyping

- Digital Logic Labs: Teach combinational logic design
- FPGA Development Boards: On-board seven-segment displays for debugging
- Embedded System Projects: Simple numerical output display

CHAPTER 9

CONCLUSION

The Binary to Seven-Segment Decoder project successfully achieved its objectives of designing a functional decoder and developing a comprehensive SystemVerilog testbench for verification.

Key Accomplishments

1. **Design Implementation:** A synthesizable, synchronous decoder was implemented in Verilog HDL, correctly mapping all 16 hexadecimal values to their corresponding seven-segment patterns.
2. **Layered Testbench Architecture:** A complete verification environment was developed using SystemVerilog, featuring transaction, generator, driver, monitor, scoreboard, and environment classes following industry-standard practices.
3. **Constrained Random Verification:** The testbench employed randomized stimulus generation with constraints, demonstrating modern verification techniques.
4. **Functional Coverage:** 100% coverage of all input values was achieved, confirming comprehensive testing.
5. **Self-Checking Mechanism:** The scoreboard's reference model enabled automatic pass/fail determination without manual inspection.
6. **Complete Verification:** All 16 input combinations produced correct outputs, with a 100% pass rate.

The project demonstrates the integration of digital design fundamentals with modern verification methodologies, providing valuable experience applicable to VLSI design and verification careers.

FUTURE SCOPE

Several enhancements can extend this project:

Design Enhancements

1. BCD-Only Mode: Add an option to display only decimal digits (0–9) with blank or error indication for inputs A–F.
2. Multiplexed Display: Extend to drive multiple seven-segment digits using time-division multiplexing.
3. Brightness Control: Add PWM-based segment brightness adjustment.
4. Common-Anode/Cathode Selection: Parameterize the design to support both display types.
5. Leading Zero Suppression: Blank leading zeros in multi-digit displays.

Verification Enhancements

1. UVM Migration: Convert the testbench to Universal Verification Methodology for industry compatibility.
2. Assertion-Based Verification: Add SystemVerilog assertions to check timing and protocol requirements.
3. Code Coverage Integration: Combine functional coverage with code coverage metrics.
4. Formal Verification: Apply formal methods to prove correctness mathematically.
5. Regression Automation: Develop scripts for automated regression testing.

Implementation Extensions

1. FPGA Implementation: Synthesize and test on physical FPGA hardware with actual seven-segment displays.
2. ASIC Synthesis: Evaluate area, power, and timing in an ASIC flow.
3. Low-Power Design: Implement clock gating and power optimization techniques.

These enhancements would further develop both the design and verification aspects, providing deeper learning opportunities and more robust solutions.

REFERENCES

- [1] M. M. Mano and M. D. Ciletti, "Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog," Pearson, 6th Edition, 2017.
- [2] S. Brown and Z. Vranesic, "Fundamentals of Digital Logic with Verilog Design," McGraw-Hill, 3rd Edition, 2014.
- [3] N. Weste and D. Harris, "CMOS VLSI Design: A Circuits and Systems Perspective," Addison-Wesley, 4th Edition, 2011.
- [4] D. Thomas and P. Moorby, "The Verilog Hardware Description Language," Springer, 5th Edition, 2002.
- [5] Xilinx Inc., "Vivado Design Suite User Guide: Logic Simulation," Xilinx Documentation, 2021.
- [6] Intel Corporation, "ModelSim User's Manual," Intel FPGA Documentation, 2020.
- [7] R. Saleh, S. Wilton, S. Mirabbasi, A. Hu, M. Greenstreet, G. Lemieux, P. Pande, C. Grecu, and A. Ivanov, "System-on-Chip: Reuse and Integration," *Proceedings of the IEEE*, vol. 94, no. 6, pp. 1050–1069, 2006.
- [8] J. L. Hennessy and D. A. Patterson, "Computer Architecture: A Quantitative Approach," Morgan Kaufmann, 6th Edition, 2019.
- [9] J. Bhasker, "A Verilog HDL Primer," Star Galaxy Publishing, 3rd Edition, 2019.